בס"ד

דו"ח פרוייקט מסכם

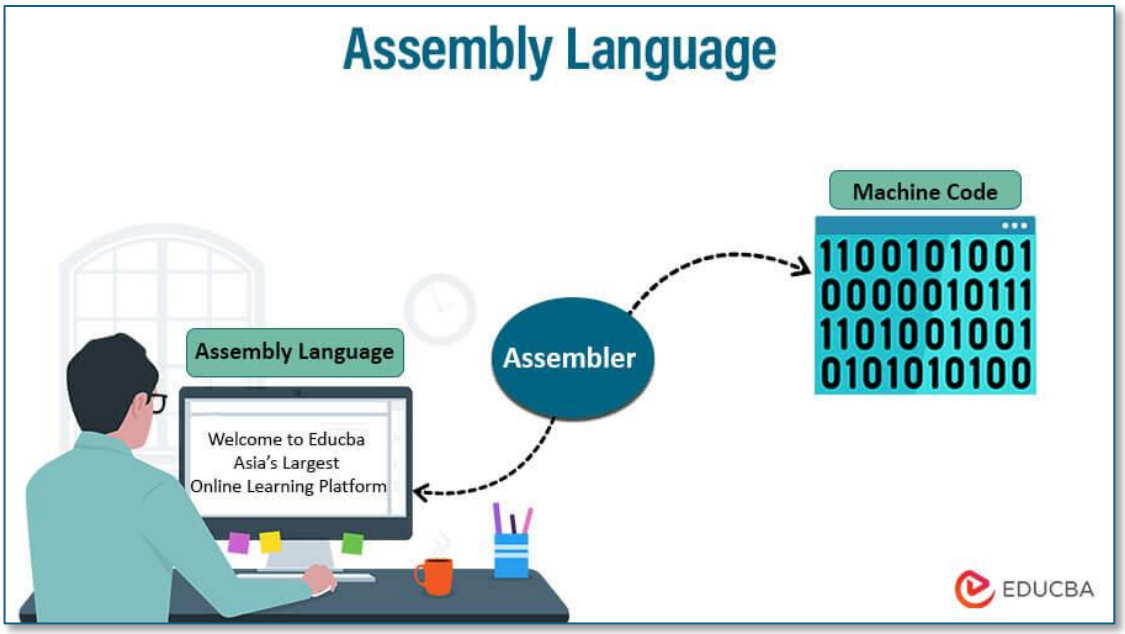
מיקרו מעבדים ושפת אסמבלר

מרצה: פרופ' רן גלס.

פרוייקט: תוכנת BEEPER'S MAZE.

מגישים: אורי מימון, ת"ז: 216383174.
נתן קרומביין, ת"ז: 341004471.

תאריך: י"ט אלול התשפ"ד, 21.9.2024.



בשם ה' נעשה ונצליח!

בס"ד

<u>מטרת הפרוייקט</u> הינה לייצר משחק בו הדמות הראשית צריכה לצאת ממבוך בזמן שדמות אחרת רודפת אחריה.

באופן ספציפי, המשחק , הקרוי BEEPER'S MAZE, מכניס אותנו למעין עלילה בה ישנה דמות ראשית- 'הגיבור' הקרוי 'ביפר', שמטרתו לצאת מהמבוך. הבעיה היא, שדמות אחרת- 'הרשע' הקרוי 'בופר', רודף אחריו.

השחקן בעצם שולט על תזוזת הדמות באמצעות מקשי D,S,A,W (כאשר כמובן W בשביל לזוז מעלה, S בשביל לזוז מטה, A בשביל לזוז שמאלה ו- D בשביל לזוז ימינה).

הדמות הראשית מחליפה כל כמה זמן את צבעה לירוק, לזמן מוגבל, ורק בזמן זה ניתן להזיז את הדמות. אם השחקן לא הספיק להזיז את ביפר, או שלא הזיזה בכיוון הנכון, בופר מתקדם צעד נוסף אל ביפר.

אם בופר מתקרב יותר מידי לביפר, הוא נהפך לבלתי נראה אך עדיין עוקב אחרי ביפר.

המטרה היא לצאת מהמבוך בכמה שפחות צעדים. ברגע שביפר יוצא מהמבוך – השחקן ניצח, אחרת, אם בופר תפס אותו – השחקן הפסיד.

<u>התוכנה בנוייה</u> בצורה הבאה:

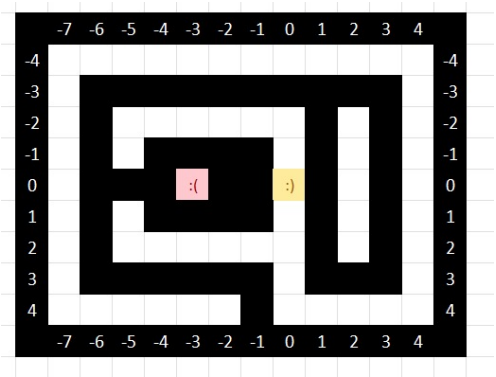
ראשית, אנחנו מגדירים את הקבועים שלנו (כמות השורות/עמודות הקיימות, מיקום הכותרת העליונה), את גודל המחסנית (קבענו אותו לh400 כי נעשה שימוש תדיר במחסנית בעקבות כך שיש קריאות להרבה פונקציות) ואת המשתנים שנרצה לשמור בזיכרון (ממחרוזות ועד למיקומי המשבצות במבוך וכו').

לאחר מכן, ישנו הקוד עצמו. הקוד בנוי בצורה הבאה:

1. אנו פותחים בכך שאנחנו מגדירים את סגמנט הזיכרון וכן סיגמנט נוסף לזיכרון המסך. בנוסף, אנחנו קוראים לפונקציה שמנקה את המסך ולפונקציה שמדפיסה את מסך הבית. לאחר מכן, אנחנו משתמשים בלולאה שבודקת האם נלחץ תו הSPACE. אם לא, אנחנו ממתינים עד שילחץ. אם נלחץ, ממשיכים לעבר המבוך.

2. לאחר מכן, אנחנו קוראים לפונקציה שמנקה את המסך, לפונקציה שמדפיסה את המבוך וקריאה לפונקציה שתדפיס כל דמות במיקומם ההתחלתי.

3. אחרי שהכול מוכן, אנחנו נכנסים ללולאת המסך (gameLoop) .
בחלק זה ישנם כמה אתגרים שהצלחנו לפתור:
   - מעקב של בופר- כדי שבופר ידע לאן לנוע, אנחנו מגדירים מערך בזיכרון הקרוי moveMemory. מערך זה שומר את הארבעה צעדים האחרונים שביפר עשה. באמצעות קריאה ושינוי המערך, הצלחנו להתגבר על אתגר זה.
   - תכנון המבוך- ראשית, ציירנו את המבוך שרצינו בxcel: לאחר מכן, חילקנו את הקירות לאנכיים ואופקיים בהתאם למה שהיה נוח. ואז, שמרנו בזיכרון את המיקום ההתחלתי והסופי ועם לולאה מילאנו את כל השורה, כאשר זה אופקי קופצים כל פעם ב-2 ועבור אנכי קופצים ב-160.



- מחוונים לשחקן- רצינו להוסיף הדפסות למסך בהתאם לצעד של השחקן כמו: 'Bad!', 'Wow!', 'Good job!', 'Amazing'. כדי לעשות זאת כתבנו פונקציה שבהתאם לערך של cx מדפיסה למסך את התגובה המתאימה.

כעת- בחזרה לקוד. בכל איטרציה קורות הפעולות הבאות:

- הפיכת ביפר לצהוב, ע"י קריאה לפונקציה המדפיסה אותו.
- המתנה של שתי שניות שבמהלכן אנחנו מוודאים שאם השחקן מכניס תו אז הוא יקלט אך לא יעשה בו דבר וזאת על מנת שלא ישאר ב-pipe להמשך.
- הפיכת ביפר לירוק, ע"י קריאה לפונקציה המדפיסה אותו.
- מדידת השנייה בעוד 2 שניות מרגע זה ושמירתה ב- dl.
- קריאה לפונקציה readKeyboard, האחראית לבדוק האם השחקן הכניס תו כל עוד לא עברו 2 שניות. אם עברו, ביפר נשאר במקום אך בופר מתקדם לעברו. אם הוא הספיק, אנחנו מעבירים את הטיפול בתו הנקלט בהתאם לערכו לפונקציות ייעדויות: moveUp, moveDown, moveRight, moveLeft. פונקציות אלה בודקות האם התו שהוקש חוקי – כלומר אם ביפר יזוז לפיו האם הוא יתקע בקיר. אם הוא חוקי הן מעדכנות את המיקום של ביפר בזיכרון. לאחר ביצוע הפונקציות האלה, מבוצעת הפונקציה shiftMoveMemory האחראית להזיז את הערכים במערך moveMemory אחד שמאלה ולדחוף 0 מימין. בכך אנחנו מוודאים שבופר יבצע את הצעד המתאים בפעם הבאה.
- לאחר קליטת התו, אנחנו בודקים האם המיקום של ביפר זהה למיקום של בופר, כלומר, האם הוא נתפס על ידו, ואם כן אנחנו עוברים להדפסת מסך ההפסד. אם לא, אנחנו בודקים האם ביפר הגיע למיקום במסך שמתאר את המוצא. אם כן, אנחנו עוברים להדפסת מסך הניצחון.
- לאחר טיפול בתזוזה של ביפר, אנחנו עוברים לתזוזה של בופר. אנחנו קוראים לפונקציה moveBopper האחראית להזזת בופר ע"פ הערך השמור בתחילת המערך moveMemory. בכך בופר בעצם עושה את הצעד שעשה ביפר שלוש מהלכים אחורה ובכך אנחנו מקבלים מעקב של בופר על ביפר.
- שוב אנחנו מבצעים בדיקה אם ביפר נתפס ע"י בופר. אם כן, עוברים למסך ההפסד. אחרת, חוזרים על הלולאה.

- לבסוף, אם ניצחנו או הפסדנו, אנחנו מחזירים את השליטה למערכת ההפעלה ובכך מסיימים את התוכנית.

<u>מרכיבים ועקרונות שנלמדו בקורס שהתוכנה מכילה</u>

- מיעון זיכרון וקודי פקודה (הרצאות 3,4):
השתמשנו במהלך הקוד שלנו הן ברגיסטרים והן בזיכרון. עברנו והעברנו מידע ביניהם באמצעות מיעוני זכרון ע"י פקודת MOV.

- שימוש בפקודות אריתמטיות/לוגיות ופקודות בקרה (הרצאות 5,6):
במהלך התוכנית השתמשנו רבות בפקודות חישוב ארתמטיות מלחבר בין שני רגיסטרים עד לשימוש ב-AND על מנת לקבל רק ביטים מסויימים. בנוסף, השתמשנו בפקודות בקרה על מנת לבצע קפיצות מותנות, מימוש if-else, לולאות ועוד.

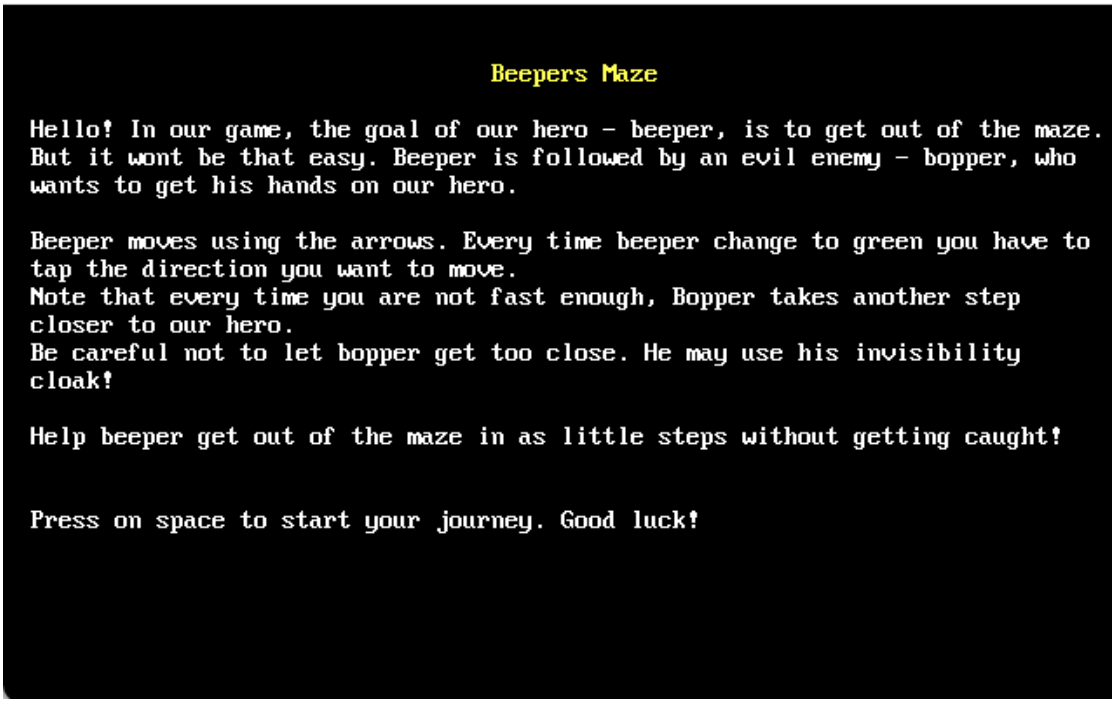
- שימוש במחסנית ותתי-שגרות (הרצאות 7,8):
במהלך התוכנית השתמשנו במחסנית על מנת לשמור זמנית תוכן של רגיסטרים, וכן שמירת נתונים בעת מעבר לתתי-שגרות. בנוסף, השתמשנו לאורך כל הקוד בתתי-שגרות. הפכנו קטע קוד לתת-שגרה כאשר צריך היה לחזור על אותה פעולה מספר פעמים או כאשר היה ניתן להתייחס אליהם כמקשה אחת ובכך להקל על נראות הקוד. יתר על כן, החלוקה לתתי-שגרות הקלה עלינו בכך שעבדנו על הקוד שני אנשים. אציין כאן, שבכל תת-שגרה, הכנסנו בתחילתה את כל הרגיסטרים שלא נרצה שישתנו מחוץ לתת השגרה למחסנית, ובסוף התת-שגרה הוצאנו אותם מהמחסנית. בכך ערכם לא השתנה בחוץ.

- שימוש בקלט/פלט ופסיקות (הרצאות 9,10):
במהלך התוכנית השתמשנו במקלדת כהתקן קלט והשתמשנו בפקודות IN/OUT על מנת לקרוא ולכתוב מהפורט המתאים. התקן נוסף בו השתמשנו היה ה-RTC בו השתמשנו כדי לדעת מהו הזמן המדוייק. השתמשנו בשיטת משאל כשיטה לגישה להתקנים הנ"ל. מבחינת פסיקות, השתמשנו בפסיקות DOS במהלך התוכנית. לדוגמה, בסיום התוכנית וכן שרצינו להדפיס למסך את כמות הצעדים שלקחו לשחקן כדי לנצח.

הוראות הפעלה

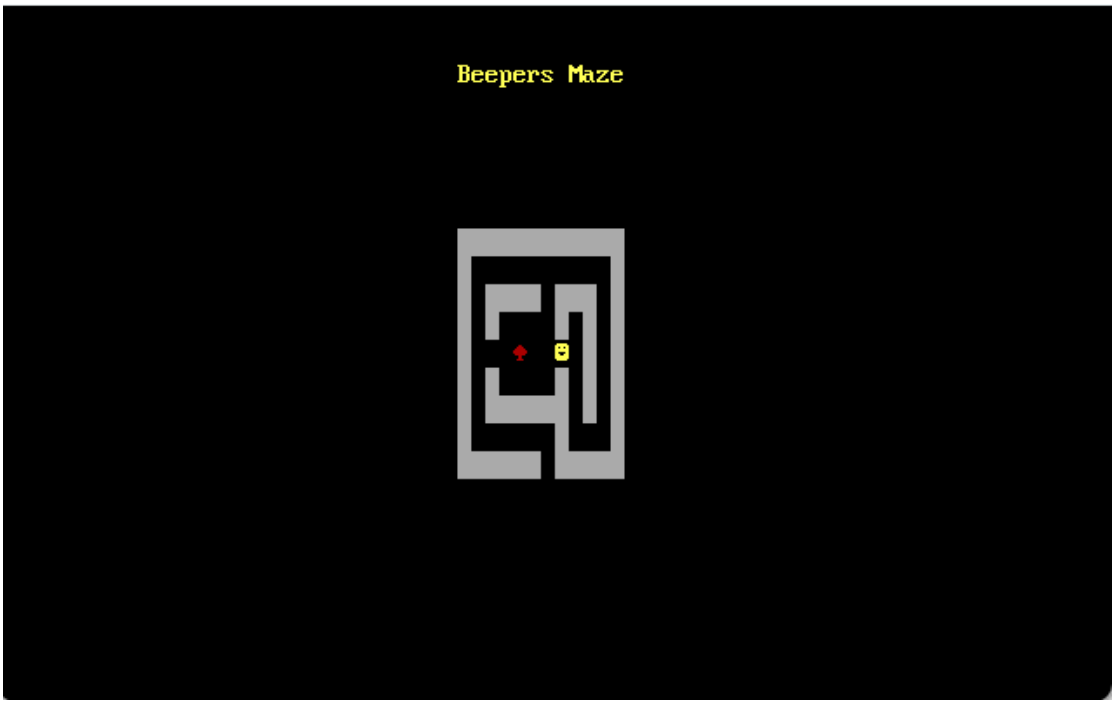
ראשית, מכניסים לDOSBOX את **maze.exe** ובכך מריצים את התוכנה.

לאחר הרצת התוכנה, יופיע על המסך מסך הבית, המספר את העלילה:

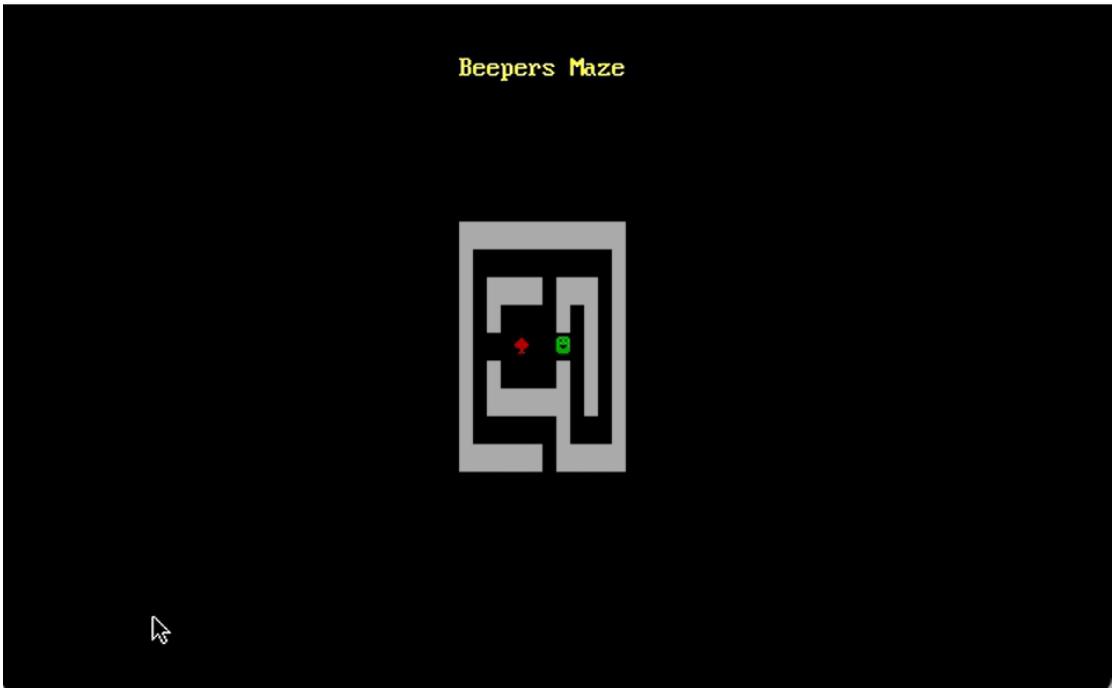


לחיצה על מקש הרווח (space), תוביל אותנו לתחילת המשחק.

המשחק מתחיל בכך ששני הדמויות שלנו נמצאות במבוך, במרחק התחלתי של 3 צעדים ביניהם:



כאשר ביפר משנה לירוק עלינו ללחוץ על הכיוון בו נרצה לזוז:



מטרתנו היא להוביל את ביפר לצאת מהמבוך בכמה שפחות צעדים מבלי להיתפס.
אם הצלחנו במשימה יופיע מסך הניצחון הבא הכולל את כמות הצעדים שלקח לשחקן כדי להגיע לניצחון:

אחרת, כלומר אם נתפסנו לפני שהצלחנו לצאת, יופיע מסך ההפסד הבא:

קישור לסרטון שהעלנו של פעולת המשחק ניתן למצוא כאן.

באגים ופתרונתיהם

במהלך כתיבת התוכנה נתקלנו בבאגים רבים אך הצלחנו לגבור עליהם ונפרט על קצתם.

- באג בייצוג 'זמן היעד':
במהלך התוכנית נדרשנו למדוד כמות זמן. לשם כך השתמשנו בRTC כדי לקבל את מספר השניות כעת. לאחר מכן הוספנו לו את הזמן הרצוי ובדקנו בלולאה האם הגענו לזמן הזה. על כל פנים אמור לעבוד אך לא עבד. גילינו לאחר מכן RTC מחזיר זמן בפורמט של BCD. לכן יש לטפל במקרי קצה ואכן ביצענו זאת.

- באג בתזוזת ביפר:
באג נוסף שנתקלנו בו היה בתזוזת בופר לפי הזיכרון. לפעמים המיקום שלו היה משתגע. לאחר חקירת הנושא הבנו שהבעיה נבעה מכך שכאשר ביפר לא זז היה נכתב ערך זבל לזיכרון של בופר ואז הקוד להזיז את בופר לא ידע להתמודד עם זה. תיקנו את זה על ידי קביעת ערך מספרי להכניס לזיכרון של בופר כאשר ביפר לא זז ואז כאשר אנחנו קוראים מהזיכרון אנחנו לא מתייחסים למספר זה ופשוט עוברים למספר התקין הבא.